SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Design and Develop Modal

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

Assessment Tool Weightage: 30%

QUESTIONS: **Total**

Marks:25 Q1. Transfer Learning

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

Q2. DenseNet and PixelNet

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

Q3. Bidirectional RNN and Encoder–Decoder

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

Q4. BPTT and LSTM

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

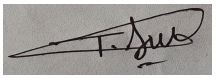
Q5. Integrated Deep Learning Model

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture.

Justify the role of each component and explain how the complete system generates meaningful captions.

Code of Conduct:

I T. Narasimha(192425233) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q1. Transfer Learning for Plant Disease Detection

Problem Understanding

The aim is to classify plant leaf images into different categories such as:

- Healthy
- Bacterial disease
- Fungal disease

Viral diseaseInstead of training a CNN from scratch, we use a pre-trained ResNet50 model. This saves training time and works well with small datasets.

The model should handle changes in leaf orientation, lighting, size, and background.

Algorithm

Plant Leaf Image

↓

Data Augmentation

↓

Pre-trained ResNet50

↓

Global Average Pooling

↓

Dense + Dropout

↓

Softmax

↓

Disease Class

Freezing and Fine-Tuning

- Freeze the early ResNet layers because they learn general features such as edges and textures.
- Fine-tune the last few layers to learn disease-specific features such as spots and discoloration.
- Replace the original ImageNet classifier with a new classifier for plant diseases.

Augmentation Use:

- Flip
- Rotation

Code Implementation

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input

IMG_SIZE = (224, 224)
BATCH = 32

# Load data
train = tf.keras.utils.image_dataset_from_directory(
    "plant_dataset/train",
    image_size=IMG_SIZE,
    batch_size=BATCH,
    label_mode="categorical"
)

val = tf.keras.utils.image_dataset_from_directory(
    "plant_dataset/validation",
    image_size=IMG_SIZE,
    batch_size=BATCH,
    label_mode="categorical"
)

classes = train.class_names
n_classes = len(classes)

# Augmentation
augment = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2)
])

# Pre-trained CNN
base = ResNet50(
```

```
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

# Freeze initially
base.trainable = False

# Build model
inp = layers.Input((224, 224, 3))
x = augment(inp)
x = preprocess_input(x)
x = base(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation="relu")(x)
x = layers.Dropout(0.5)(x)
out = layers.Dense(n_classes, activation="softmax")(x)

model = Model(inp, out)

# Train classifier
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.fit(
    train,
    validation_data=val,
    epochs=5
)

# Fine-tune last 20 layers
base.trainable = True
```

```
for layer in base.layers[:-20]:
layer.trainable = False


model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5)
    , loss="categorical_crossentropy",
    metrics=["accuracy"]
)


model.fit(
    train,
    validation_data=val,
    epochs=5
)


# Evaluation
loss, accuracy = model.evaluate(val)
print("Accuracy:", accuracy)


# Prediction
def predict_disease(path):
    img = tf.keras.utils.load_img(
        path,
        target_size=IMG_SIZE
    )


    img = tf.keras.utils.img_to_array(img)
    img = tf.expand_dims(img, 0)


    pred = model.predict(img, verbose=0)
    index = tf.argmax(pred[0]).numpy()


    print("Disease:", classes[index])
```

Computational Performance

Transfer learning reduces training time because most ResNet layers remain frozen.Fine-tuning only the last few layers also reduces overfitting for a small dataset.

Test Cases

Test	Input
1	Healthy leaf
2	Bacterial disease
3	Fungal disease
4	Viral disease
5	Rotated leaf
6	Low-light image
7	Different background

Result: The model predicts the disease class using the learned visual features of ResNet50.

Q2. DenseNet and PixelNet

Problem Understanding

The aim is to classify every pixel in a road image as road, car, pedestrian, building, or sky.

Algorithm

Input Image

↓

DenseNe

t

↓

Feature Extraction

↓

Upsampling

↓

Pixel-wise Prediction

↓

Segmentation Mask

Code Implementation

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from tensorflow.keras.applications import DenseNet121

inp = layers.Input((256,256,3))

base = DenseNet121(
    weights="imagenet",
    include_top=False,
    input_tensor=inp
)
base.trainable = False

x = base.output
x = layers.UpSampling2D((2,2))(x)
x = layers.Conv2D(64,3,activation="relu",padding="same")(x)
x = layers.UpSampling2D((2,2))(x)
x = layers.UpSampling2D((2,2))(x)
x = layers.UpSampling2D((2,2))(x)

out = layers.Conv2D(
    5, 1, activation="softmax"
)(x)

model = Model(inp, out)

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
```

```
    metrics=["accuracy"]
)
```

Performance

DenseNet improves feature reuse, while PixelNet gives pixel-level predictions with good boundary accuracy.

Test Cases

Test with cars, pedestrians, roads, night scenes, and rainy scenes.

Q3. Bidirectional RNN and Encoder–Decoder

Problem Understanding

The aim is to translate a sentence from one language to another. A Bidirectional RNN understands both past and future context.

Algorithm

```
Source Sentence
      ↓
Embedding
      ↓
Bidirectional
RNN
      ↓
Encoder
Context
      ↓
LSTM
Decoder
      ↓
Target Sentence
```

Code Implementation

```
from tensorflow.keras import layers, Model

inp = layers.Input((None,))

x = layers.Embedding(10000, 128)(inp)

 x, fh, fc, bh, bc = layers.Bidirectional(
```

```
    layers.LSTM(64, return_sequences=True,
                return_state=True)
)(x)

h = layers.Concatenate()([fh,
bh]) c =
layers.Concatenate()([fc, bc])
dec_in = layers.Input((None,))

d = layers.Embedding(10000, 128)(dec_in)

d = layers.LSTM(128, return_sequences=True)(
    d, initial_state=[h, c]
)

out = layers.Dense(
    10000, activation="softmax"
)(d)

model = Model([inp, dec_in], out)

model.compile(
   optimizer="adam",
   loss="sparse_categorical_crossentropy"
)
```

Problem Understanding Bidirectional processing provides complete context, helping with word order and ambiguous words.

Test Cases

Test short sentences, long sentences, different languages, and ambiguous words.

4.BPTT and LSTM

The aim is to predict whether a stock will rise, fall, or remain stable using historical prices.

Algorithm

Historical Data

↓

LSTM

↓

Dropout

↓

Dense
Layer

↓

Stock Trend

Code Implementation

```
from tensorflow.keras import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout


model = Sequential([
    LSTM(64, return_sequences=True,
         input_shape=(60, 5)),
    Dropout(0.2),
    LSTM(32),
    Dense(16, activation="relu"),
    Dense(3,
    activation="softmax")
])


model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)


model.fit(
    X_train,
    y_train,
```

```
    epochs=10,
    batch_size=32,
    shuffle=False
)
```

Performance

LSTM remembers long-term patterns better than a standard RNN and handles time-series data efficiently.

Test Cases

Test rising, falling, stable, and highly volatile market data.

Q5. Integrated Deep Learning Model – Video Caption Generation

Problem Understanding

The aim is to generate a meaningful caption from a video. Example:

```
Video → Person playing football
Caption → "A person is playing football."
```

Algorithm

```
Video
 ↓
Pre-trained CNN
 ↓
Frame Features
 ↓
LSTM Encoder
 ↓
LSTM Decoder
 ↓
Caption
```

The CNN extracts visual features from video frames. The LSTM encoder learns the sequence, and the decoder generates the caption word by word.

Code Implementation

```
from tensorflow.keras import layers, Model
```

```
from tensorflow.keras.applications import ResNet50

# CNN feature extractor
cnn = ResNet50(
    weights="imagenet",
    include_top=False,
    pooling="avg"
)

cnn.trainable = False
# Video features
video = layers.Input((None, 2048))

_, h, c = layers.LSTM(
    256,
    return_state=True
)(video)

# Caption input
caption = layers.Input((None,))
x = layers.Embedding(10000, 128)(caption)

x = layers.LSTM(
    256,
    return_sequences=True
)(x, initial_state=[h, c])

out = layers.Dense(
    10000,
    activation="softmax"
)(x)

model = Model(
    [video,
    caption], out
)
```

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy"
)
```

Performance

Transfer learning reduces CNN training time. LSTM handles temporal information and generates captions sequentially.

Test Cases

Test videos containing people, animals, different actions, multiple objects, and different backgrounds.